



INDIA.RUNS

Build what next India runs on

Team Name : rajneel-chavan

Team Leader Name : Rajneel Chavan

Problem Statement : Rank 100,000 candidates for a Senior AI Engineer role at Redrob AI – CPU-only, no external API calls, runtime under 5 minutes

Solution Overview

- A pure-Python, multi-component feature scoring system that reads 100K candidate profiles (JSONL) and outputs the top 100 ranked candidates as a CSV – with zero model loading, zero external API calls, and a runtime of 33 seconds on CPU. Six weighted components (title tier, AI-specific experience, skills quality, career content, consulting trajectory, location) are combined into a base score, then multiplied by a behavioral modifier from 7 real-time signals (open-to-work status, recency, recruiter response rate, notice period, interview completion, GitHub activity, saved by recruiters). Final scores are linearly rescaled to [0.10, 0.99] to eliminate ties.
- Traditional systems match candidates by counting AI keywords in their skills list – a trap the dataset is deliberately designed to exploit. Our system uses job title as the primary anchor: a Marketing Manager with FAISS in their skills is still ranked near zero. We scan career description text for hard-to-fake production evidence (NDCG, A/B testing, FAISS deployed, shipped to production) weighted 3.5-5x vs. generic terms. Skills quality = proficiency x endorsements x usage

JD Understanding & Candidate Evaluation

- Key JD requirements: hands-on production experience with ranking, retrieval, and recommendation systems; specific metrics knowledge (NDCG, MRR, MAP); familiarity with vector DBs (FAISS, Qdrant, Weaviate); A/B testing and shipped ML systems; India-based preferred; explicitly disqualifies all-consulting careers (TCS/Infosys/Wipro only) and keyword stuffers.
- Most important signals (in weight order): AI/ML-specific years of experience extracted from career history (0.25 weight) – not just total YoE; current job title tier (0.22) – Tier-1 targets are Search Engineer, Recommendation Systems Engineer, Applied ML Engineer, NLP Engineer; career description content (0.22) – weighted term scan for NDCG, shipped, production, FAISS, A/B test; skills quality (0.20) – proficiency x endorsements x usage duration catches keyword stuffers; consulting trajectory penalty (0.08); India location preference (0.03). Behavioral modifier (0.25-1.30x) multiplies the base score.

Ranking Methodology

- Retrieval: all 100K candidates are scored (no pre-filtering by embedding similarity). A fast-path filter exits early for non-technical titles with no AI skills – these 65K candidates get score 0.05 x behavioral modifier, skipping the full analysis. The remaining 35K candidates go through full 6-component scoring. After scoring, sort descending and take top 100.
- No ML models loaded. Pure deterministic heuristics: (1) Title tier mapping – regex match against curated Tier-1/2/3 title lists derived from the full dataset. (2) AI experience extraction – count months in roles with AI/ML title keywords (100% credit) or high-signal AI terms in descriptions (60% credit); consulting roles get 0.65x recency weight. (3) Skills quality formula: $\text{weight} \times \text{proficiency_score} \times (0.5 + 0.25 \times \text{endorse_factor} + 0.25 \times \text{duration_factor})$. (4) Career term scan – weighted dictionary of 40+ terms (NDCG=5.0, FAISS=4.5, A/B test=4.0, shipped=3.5, model=1.0); recency-weighted so most recent role counts 3x more. (5) Trajectory – penalise all-consulting, reward product-company AI progression. (6) Location – India city match.

Explainability & Data Validation

- Every candidate in the output CSV includes a machine-readable reasoning field listing: their title tier score, AI-specific years of experience, top matched career terms found in descriptions, behavioral modifier value, and whether the fast-path filter was applied. All scores are deterministic weighted sums — no black box. Every weight is documented and justified against the JD. A recruiter can audit exactly why CAND_0071974 scored 0.990 and why a Marketing Manager with AI skills scored 0.04.
- Zero LLM calls — no generative model is involved at any point during ranking. No hallucination is possible because there is no generation. All term matches reference exact strings found in career_history description fields. All title tier assignments use exact regex matching against a curated list. All behavioral signals are read directly from structured profile fields (last_active_date, recruiter_response_rate, notice_period_days, etc.) with no interpretation or inference.
- Honeypot detection fires on 5 checks: (1) Timeline impossibility — total career months > 2.2x

End-to-End Workflow

- Step 1 – Input: Read candidates.jsonl (465 MB, 100K profiles) or .gz compressed; parse JSONL line by line with no memory buffering. Step 2 – Honeypot check: 5 integrity checks per candidate; flagged profiles receive score 0.001 and are excluded from ranking. Step 3 – Fast-path filter: if non-technical title AND no AI skills in profile → assign score 0.05 x behavioral modifier and skip full analysis (covers ~65K of 100K candidates, saves ~130 seconds). Step 4 – Full feature scoring: compute 6 components (title, ai_experience, skills_quality, career_content, trajectory, location) for remaining candidates. Step 5 – Behavioral modifier: multiply base score by modifier (0.25-1.30x) derived from 7 real-time profile signals. Step 6 – Rank and rescale: sort all candidates by final score descending, take top 100, linearly rescale scores to [0.10, 0.99] to eliminate ties. Step 7 – Output: write rajneel_chavan.csv with columns candidate_id, rank, score, reasoning. Total runtime: 33 seconds on Apple M2 CPU.

System Architecture

- Input Layer: candidates.jsonl / candidates.jsonl.gz (465 MB, 100K profiles) — parsed line-by-line with Python `stdlib json` module, no memory buffering, supports both plain and gzip-compressed input.
- Scoring Engine (`rank.py` — single file, Python `stdlib` only): Fast-path filter (non-technical title + no AI skills → early exit at 0.05 x behavioral). Full scorer: `title_tier_map()` → `ai_experience_scorer()` → `skills_quality_scorer()` → `career_content_scanner()` → `trajectory_scorer()` → `location_scorer()`. Behavioral modifier: 7 signals multiplied together (range 0.25-1.30x). Honeypot detector: 5 integrity checks, flags score to 0.001.
- Output Layer: sort descending → take top 100 → linear rescale to [0.10, 0.99] → write `rajneel_chavan.csv`. Runtime: 33 seconds on Apple M2 CPU (limit: 5 minutes). Zero external dependencies — Python `stdlib` only (`json`, `csv`, `datetime`, `pathlib`, `argparse`).
- Demo Layer (optional): Streamlit app deployed at `redrob-candidate-ranker-qckdhksnfxczjr3dqmffc8.streamlit.app` — accepts JSONL upload, runs `rank.py` end-to-end, returns ranked CSV download.

Results & Performance

- Top 100 results: 100% Tier-1/2 AI/ML titles — Search Engineers, Recommendation Systems Engineers, Applied ML Engineers, Senior NLP Engineers, Machine Learning Engineers, AI Engineers, Data Scientists. Zero non-technical candidates in top 100. 87 out of 100 candidates are India-based (matching JD preference). 0 honeypot profiles in top 100. Top score: CAND_0071974 (Senior AI Engineer, 7.8 years AI experience) at 0.990. Scores range from 0.990 down to 0.102 across the top 100, with no ties — all candidates uniquely differentiated.
- Runtime: 33 seconds on Apple M2 CPU — 9x faster than the 5-minute constraint. Compute: CPU-only, no GPU, no model loading. Memory: streaming JSONL parse, no full dataset loaded into RAM simultaneously. Dependencies: zero — Python stdlib only, no pip install required. The fast-path optimisation (early exit for 65K non-technical candidates) reduced runtime from ~165 seconds to 33 seconds. The solution runs identically on any Python 3.8+ environment with no setup. Reproduce command: `python solution/rank.py -candidates ./candidates.jsonl -out`

Technologies Used

- Python 3.11 (stdlib only) – selected because the challenge requires no external dependencies at inference time; json, csv, datetime, pathlib, argparse cover all needs without any pip install. No ML framework (no PyTorch, scikit-learn, HuggingFace, spaCy) – deliberately avoided to prove that good ML engineering does not always require a model; pure feature engineering outperforms keyword-stuffed embeddings when signals are well-designed. No LLM / no API calls – aligns with the production constraint; zero hallucination risk. Custom weighted scoring algorithm – hand-tuned weights derived from careful JD analysis; multiplicative behavioral modifier from 7 real-time signals; post-ranking linear rescaling. Fast-path optimisation – reduced runtime 5x (165s to 33s) with a simple non-technical title check before full scoring. Streamlit (demo only, not used in ranking) – deployed free on Streamlit Cloud for interactive demonstration; accepts JSONL, runs rank.py, returns CSV. AI tools declared: Claude (Anthropic) for architecture discussion and code review; GitHub Copilot for autocomplete. No candidate data was fed to any LLM during

Submission Assets

- GitHub Repository: <https://github.com/Rajneel-Chavan/redrob-candidate-ranker> — contains rank.py (main ranker), README.md (full architecture docs), rajneel_chavan.csv (ranked output), submission_metadata.yaml, requirements.txt (empty — no dependencies)
- Live Demo: <https://redrob-candidate-ranker-qckdhksnfxczjr3dqmffc8.streamlit.app> — Streamlit app deployed on Streamlit Cloud; upload a JSONL file, get back a ranked CSV download; runs rank.py end-to-end
- Ranked Output CSV: rajneel_chavan.csv — 101 lines (header + 100 ranked candidates); validated by validate_submission.py; 100% Tier-1/2 AI titles, 87/100 India-based, 0 honeypots
- Reproduce: `python solution/rank.py -candidates ./candidates.jsonl -out ./rajneel_chavan.csv` — one command, no setup, ~33 seconds on any CPU



INDIA.RUNS

Build what next India runs on

THANK YOU